

설계서 작성 지침2(GPT)

◆ 랜딩 - 동렬

- Controller: `LandingController`
- Service: 없음
- Entity: 없음
- DTO: 없음

◆ 메인 - 찬용

- Controller: `MainController`
- Service: `MainService`
- Entity: `User`, `Pet`
- DTO: 없음 (`Model` 처리)

◆ 회원 - 찬용

- Controller: `AuthController`
- Service: `AuthService`
- Entity: `User`
- DTO: `LoginRequest`, `LoginResponse`, `SignupRequest`

◆ 계정 찾기 - 찬용

- Controller: `FindAccountController`
- Service: `FindAccountService`
- Entity: `User`
- DTO: 없음 또는 추후 추가

◆ 대시보드 - 현규

- Controller: `DashboardController`
- Service: `DashboardService`
- Entity: `Pet`, `State`, `EventLog`, `AlertLog`, `ActionLog`, `Device`
- DTO: 없음 (`Model` 처리)

◆ 미디어 - 동렬

- Controller: `MediaController`
- Service: `MediaService`
- Entity: `MediaLog` , `CameraLog` , `Device` , `Pet`
- DTO: 없음 (`Model` 처리)

◆ 히스토리(펫관리) - 도헌(동렬)

- Controller: `HistoryController`
- Service: `HistoryService`
- Entity: `AlertLog` , `ActionLog` , `EventLog` , `Pet`
- DTO: 없음 (`Model` 처리)

◆ 환경설정 - 도헌(현규)

- Controller: `SettingsController`
- Service: `SettingsService` , `PetService`
- Entity: `Pet` , `PetBaselineSetting` , `UserNotificationSetting` , `EventThresholdSetting` , `EnvironmentLog` , `User`
- DTO: `PetCreateRequest` , `PetUpdateRequest` , `ThresholdSettingRequest`

◆ 관리자 - 현규

- Controller: `AdminController`
- Service: `AdminService`
- Entity: `User` , `Pet` , `Device` , `EnvironmentLog`
- DTO: 없음 또는 추후 추가

◆ 시뮬레이션 - 도헌(찬용)

- Controller: `SimulationController`
 - Service: `SimulationService` , `EventEvaluationService` , `AlertService` , `ActionService`
 - Entity: `WearableLog` , `FeederLog` , `WaterLog` , `CameraLog` , `DeviceAnalysisLog` , `State` , `EventLog` , `RiskScoreLog` , `AlertLog` , `ActionLog` , `BehaviorCode` , `EventCode`
 - DTO: `SimulationRunResult`
-

B. 따로 기억하면 좋은 보조 짝궁

이건 페이지 1개에만 묶이지 않고 옆에서 지원하는 클래스입니다.

- `PetService` ↔ `Pet` , `PetBaselineSetting`
- `EventEvaluationService` ↔ `State` , `EventLog` , `RiskScoreLog` , `BehaviorCode` , `EventCode`
- `AlertService` ↔ `AlertLog`
- `ActionService` ↔ `ActionLog`

즉 이들은 독립 페이지 서비스라기보다, 시뮬레이션/대시보드/히스토리 뒤에서 재사용되는 보조 서비스로 보면 됩니다.

C. 문서에 적을 때의 기준

클래스 설계서에는 아래 느낌으로 적으면 깔끔합니다.

- **페이지 중심 클래스:** Controller + 대표 Service
- **DB 중심 클래스:** Entity
- **입력값이 분명한 것만 DTO**
- 타임리프 화면 조회는 **응답 DTO 없이** `Model` 처리라고 적기

이렇게 하면 **API 기능이 안 빠지고**, 클래스 수도 불필요하게 늘지 않습니다.